

Санкт-Петербургский политехнический университет Петра
Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

П Р А К Т И Ч Е С К А Я Р А Б О Т А № 4

Исследование и демонстрация тестирования идиомпотентности
и повторяемости запросов в REST API на Java (Spring Boot)

по дисциплине «Методы тестирования программного обеспечения»

Выполнил
студент гр. 5130903/30303

В. А. Плахотников

Руководитель
старший преподаватель
ВШ ПИ

В. А. Пархоменко

Содержание

1	Введение	3
1.1	Актуальность	3
1.2	Постановка проблемы	3
1.3	Задачи	3
2	Методы	4
2.1	Математическая постановка задачи идемпотентности	4
2.2	Классификация HTTP-методов по идемпотентности	5
2.3	Паттерн Idempotency-Key	6
2.4	Optimistic Locking как механизм идемпотентности	6
2.5	Псевдокод алгоритма проверки идемпотентности	7
2.6	Пошаговое выполнение алгоритма на иллюстративных данных	8
2.7	Типы организации контроллеров в Spring	9
2.7.1	@RestController (аннотационный подход)	9
2.7.2	Functional Endpoints (WebMvc.fn)	10
2.7.3	Spring Data REST	10
2.8	Средства тестирования	10
3	Результаты	11
3.1	Описание разработанного программного обеспечения	11
3.1.1	Структура проекта	11
3.1.2	Схема базы данных	12
3.1.3	Компоненты механизма идемпотентности	12
3.1.4	Профили конфигурации	12
3.2	Результаты тестирования (профиль idempotent)	13
3.3	Тест-кейсы по контроллерам	13
3.3.1	@RestController (OrderIdempotencyTest)	13
3.3.2	Functional Endpoints (PaymentIdempotencyTest)	14
3.3.3	Spring Data REST (ProductIdempotencyTest)	14
3.3.4	Импорт данных (DataImportTest)	14
3.4	Результаты профиля non-idempotent	15
4	Обсуждение	15
4.1	Сравнение подходов к обеспечению идемпотентности	15
4.2	Анализ результатов	16
4.3	Идемпотентность как конвенция: почему нельзя «всё делать через GET»	16
4.4	Оптимистичная блокировка и границы её применимости для идемпотентности	18
4.5	Значимость Testcontainers в тестировании идемпотентности .	19
4.6	Рекомендации для production	20

5	Заключение	20
6	Инструкция по установке и запуску	21
6.1	Требования	21
6.2	Запуск приложения	21
6.3	Запуск тестов	22
6.4	Пример использования API	22
	Приложение А. Листинги исходного кода	24

1 Введение

1.1 Актуальность

В современных распределённых системах REST API является основным способом взаимодействия между сервисами. При обработке HTTP-запросов неизбежно возникают ситуации, когда запрос может быть отправлен повторно: из-за сетевых таймаутов, retry-политик, дублирования на уровне балансировщиков нагрузки или ошибок клиента. Без механизмов идемпотентности повторная обработка запроса приводит к критическим последствиям: дублированию платежей, двойному списанию средств, созданию дубликатов записей в базе данных [1].

Тестирование идемпотентности особенно важно в финансовых и e-commerce системах, где повторная обработка транзакции приводит к прямым финансовым потерям. Компании, такие как Stripe, реализуют паттерн Idempotency-Key как обязательный элемент API [2].

1.2 Постановка проблемы

Проблема заключается в необходимости обеспечения корректного поведения REST API при повторных запросах. HTTP-спецификация [3] определяет, какие методы должны быть идемпотентными (GET, PUT, DELETE), а какие — нет (POST). Однако на практике реализация идемпотентности требует дополнительных механизмов на уровне приложения.

Цель данной работы — исследовать и продемонстрировать методы тестирования идемпотентности REST API на примере Spring Boot приложения с тремя кардинально различными типами организации контроллеров.

1.3 Задачи

1. Формализовать понятие идемпотентности HTTP-запросов математически.
2. Разработать REST API с тремя различными типами контроллеров Spring:
 - аннотационный `@RestController` (по образцу Spring Guides [4]),
 - функциональные эндпоинты `RouterFunction` (по образцу Spring Framework [5]),
 - автогенерируемые эндпоинты Spring Data REST [6].
3. Реализовать механизмы обеспечения идемпотентности (Idempotency-Key, optimistic locking, unique constraints).

4. Разработать набор интеграционных тестов с использованием Testcontainers [7].
5. Продемонстрировать поведение системы без механизмов идемпотентности через отдельный Spring-профиль.

2 Методы

2.1 Математическая постановка задачи идемпотентности

Определение. Функция $f : X \rightarrow X$ называется *идемпотентной*, если для любого $x \in X$:

$$f(f(x)) = f(x) \quad (1)$$

В контексте HTTP API определим:

- S — множество состояний системы (совокупность данных в БД);
- R — множество HTTP-запросов;
- $O : S \times R \rightarrow S \times \text{Response}$ — операция обработки запроса;
- $\pi_1(O(s, r))$ — проекция на новое состояние.

Определение (идемпотентность HTTP-метода). HTTP-метод m называется идемпотентным, если для любого состояния $s \in S$ и запроса r с методом m :

$$\pi_1(O^n(s, r)) = \pi_1(O(s, r)), \quad \forall n \geq 1 \quad (2)$$

где O^n обозначает n -кратное применение операции, при этом каждое следующее применение использует состояние, полученное на предыдущем шаге:

$$O^1(s, r) = O(s, r), \quad O^{n+1}(s, r) = O(\pi_1(O^n(s, r)), r) \quad (3)$$

Свойства. Согласно RFC 9110 [3]:

- **Идемпотентные методы:** GET, HEAD, PUT, DELETE, OPTIONS
- **Неидемпотентные методы:** POST, PATCH
- **Safe (безопасные) методы:** GET, HEAD, OPTIONS — не изменяют состояние

Формально, для safe-метода m :

$$\pi_1(O(s, r)) = s, \quad \forall s \in S, r \text{ с методом } m \quad (4)$$

Из этого следует, что каждый safe-метод является идемпотентным, но не наоборот: DELETE изменяет состояние при первом вызове (удаляет ресурс), однако повторные вызовы не изменяют его далее — ресурс уже отсутствует.

Задача. Для метода POST, который не является идемпотентным по определению, необходимо обеспечить идемпотентность через дополнительные механизмы. Обозначим k — ключ идемпотентности. Тогда модифицированная операция O' :

$$O'(s, r, k) = \begin{cases} O(s, r), & \text{если } k \notin \text{KeyStore}(s) \\ (s, \text{CachedResponse}(k)), & \text{если } k \in \text{KeyStore}(s) \end{cases} \quad (5)$$

что гарантирует: $\pi_1(O'^n(s, r, k)) = \pi_1(O'(s, r, k)), \quad \forall n \geq 1$.

Доказательство. Пусть $s_1 = \pi_1(O'(s, r, k))$. При первом вызове $k \notin \text{KeyStore}(s)$, поэтому выполняется $O(s, r)$ и ключ k регистрируется в хранилище. При втором вызове $k \in \text{KeyStore}(s_1)$, поэтому состояние не изменяется: $\pi_1(O'(s_1, r, k)) = s_1$. По индукции, для любого $n \geq 1$: $\pi_1(O'^n(s, r, k)) = s_1 = \pi_1(O'(s, r, k))$. $\square$

2.2 Классификация HTTP-методов по идемпотентности

Спецификация HTTP (RFC 9110 [3]) определяет свойства методов следующим образом:

Таблица 1: Свойства HTTP-методов (RFC 9110)

Метод	Safe	Идемп.	Тело	Семантика
GET	Да	Да	Нет	Получение представления ресурса
HEAD	Да	Да	Нет	Получение заголовков без тела
PUT	Нет	Да	Да	Полная замена состояния ресурса
DELETE	Нет	Да	Нет	Удаление ресурса
POST	Нет	Нет	Да	Создание или произвольная обработка
PATCH	Нет	Нет	Да	Частичное обновление ресурса

Различие между PUT и PATCH имеет принципиальное значение для идемпотентности. PUT заменяет ресурс целиком, поэтому повторный вызов с теми же данными не меняет состояние. PATCH применяет дельту: например, операция «увеличить количество на 1» при повторном применении даст другой результат, поэтому PATCH не является идемпотентным в общем случае.

2.3 Паттерн Idempotency-Key

Согласно IETF-стандарту [8], клиент передаёт уникальный ключ в заголовке **Idempotency-Key**. Сервер сохраняет этот ключ вместе с кэшированным ответом. При повторном запросе с тем же ключом сервер возвращает кэшированный ответ без повторного выполнения операции.

Данный паттерн широко применяется в production-системах. В частности, платёжная система Stripe требует заголовков **Idempotency-Key** для всех POST-запросов, связанных с финансовыми операциями [2]. Это предотвращает дублирование платежей при retry.

Жизненный цикл ключа идемпотентности:

1. **Генерация** — клиент генерирует UUID перед первым запросом.
2. **Регистрация** — сервер сохраняет ключ в хранилище (БД) до обработки запроса.
3. **Обработка** — выполняется бизнес-логика, результат кэшируется.
4. **Повторное использование** — при повторном запросе возвращается кэш.
5. **Истечение** — по прошествии TTL (обычно 24 часа) ключ удаляется.

2.4 Optimistic Locking как механизм идемпотентности

Для метода PUT идемпотентность может быть усилена через *optimistic locking* [9]. Каждая сущность содержит поле версии v . При обновлении клиент передаёт текущую версию, и сервер проверяет совпадение:

$$\text{Update}(s, r, v) = \begin{cases} (s', 200 \text{ OK}), & \text{если } v = \text{CurrentVersion}(s) \\ (s, 409 \text{ Conflict}), & \text{если } v \neq \text{CurrentVersion}(s) \end{cases} \quad (6)$$

В Spring Data REST это реализуется через аннотацию **@Version**: Hibernate автоматически добавляет проверку версии в SQL-запрос UPDATE, а Spring генерирует HTTP-заголовок ETag из значения версии.

Клиент передаёт ETag через заголовок *If-Match*, и при несовпадении версий сервер возвращает **412 Precondition Failed**.

Это предотвращает проблему *lost update* — ситуацию, когда два клиента одновременно читают одну версию ресурса, модифицируют его и сохраняют, при этом изменения первого клиента теряются.

2.5 Псевдокод алгоритма проверки идемпотентности

Алгоритм 1 Проверка идемпотентности HTTP-запроса

Вход: HTTP-запрос *request*, хранилище ключей *KeyStore*

Выход: HTTP-ответ *response*

```
1: Функция CHECKIDEMPOTENCY(request)
2:   если request.method  $\neq$  POST тогда
3:     вернуть PROCESSREQUEST(request)       $\triangleright$  GET/PUT/DELETE
        идемпотентны
4:   конец если
5:   key  $\leftarrow$  request.headers["Idempotency-Key"]
6:   если key =  $\emptyset$  тогда
7:     вернуть 400 Bad Request       $\triangleright$  Ключ обязателен для POST
8:   конец если
9:   cached  $\leftarrow$  KeyStore.find(key)
10:  если cached  $\neq \emptyset$  и cached.response  $\neq \emptyset$  тогда
11:    вернуть cached.response       $\triangleright$  Повторный запрос —
        кэшированный ответ
12:  конец если
13:  если cached  $\neq \emptyset$  и cached.response =  $\emptyset$  тогда
14:    вернуть 409 Conflict           $\triangleright$  Запрос уже обрабатывается
15:  конец если
16:  KeyStore.register(key, request.method, request.path)
17:  response  $\leftarrow$  PROCESSREQUEST(request)
18:  KeyStore.saveResponse(key, response)
19:  вернуть response
20: конец Функция
```

Сложность алгоритма. Поиск ключа в хранилище выполняется за $O(1)$ при использовании индекса (B-tree) на поле *idempotency_key*. Регистрация и сохранение ответа — по одной операции INSERT/UPDATE. Общая сложность: $O(1)$ дополнительных операций к каждому POST-запросу.

Обработка конкурентных запросов. Строка 15 алгоритма предотвращает гонку (race condition): если два запроса с одним ключом поступают одновременно, только первый успешно регистрирует

ключ (благодаря UNIQUE constraint в БД), второй получит ошибку `DataIntegrityViolationException` и вернёт 409 Conflict.

2.6 Пошаговое выполнение алгоритма на иллюстративных данных

Сценарий: клиент создаёт заказ (POST `/api/orders`) и из-за таймаута повторяет запрос.

Входные данные:

- Запрос: POST `/api/orders`, тело: `{description: "Ноутбук amount: 89999.99"}`
- Заголовок: `Idempotency-Key: abc-123`
- Начальное состояние: таблица `orders` пуста, `KeyStore` пуст

Первый запрос (шаг за шагом):

1. Метод = POST $\Rightarrow$ проверяем Idempotency-Key (строка 2 алгоритма)
2. $key = \text{"abc-123"} \neq \emptyset \Rightarrow$ продолжаем (строка 6)
3. $KeyStore.find(\text{"abc-123"}) = \emptyset \Rightarrow$ ключ новый (строка 9)
4. $KeyStore.register(\text{"abc-123"}, \text{POST}, \text{"api/orders"})$ (строка 17)
5. Выполняем `ProcessRequest`: INSERT INTO `orders` $\Rightarrow$ заказ создан (`id=e7f1...`) (строка 18)
6. $KeyStore.saveResponse(\text{"abc-123"}, 201, \text{"{id: e7f1...}"})$ (строка 19)
7. Ответ: 201 Created, тело: `{id: "e7f1... description: "Ноутбук"}`

Повторный запрос (retry после таймаута):

1. Метод = POST $\Rightarrow$ проверяем Idempotency-Key (строка 2)
2. $key = \text{"abc-123"} \neq \emptyset \Rightarrow$ продолжаем (строка 6)
3. $KeyStore.find(\text{"abc-123"}) \neq \emptyset, cached.response \neq \emptyset$ (строка 10)
4. Возвращаем кэшированный ответ: 201 Created, тело: `{id: "e7f1..."}` (строка 11)
5. **Заказ НЕ дублируется** — в таблице по-прежнему одна запись

Трассировка состояний:

Таблица 2: Пошаговая трассировка алгоритма на иллюстративных данных

Шаг	Действие	KeyStore	orders
0	Начальное состояние	$\emptyset$	0 записей
1	Получен key="abc-123"	$\emptyset$	0 записей
2	find("abc-123") = $\emptyset$	$\emptyset$	0 записей
3	register("abc-123")	{abc-123: pending}	0 записей
4	INSERT INTO orders	{abc-123: pending}	1 запись
5	saveResponse("abc-123", 201)	{abc-123: 201, body}	1 запись
6	Retry: key="abc-123"	{abc-123: 201, body}	1 запись
7	find("abc-123") $\neq \emptyset$	{abc-123: 201, body}	1 запись
8	return cached(201)	{abc-123: 201, body}	1 запись

2.7 Типы организации контроллеров в Spring

В данной работе используются три кардинально различных подхода к организации REST-контроллеров в Spring Framework (таблица 3).

Таблица 3: Сравнение типов контроллеров Spring

Характеристика	@RestController	Functional Endpoints	Spring REST	Data
Стиль	Аннотационный	Функциональный	Автогенерация	
Маршрутизация	@GetMapping, @PostMapping	RouterFunction	Автоматическая	
Обработчик	Метод контроллера	HandlerFunction	Репозиторий	
Бизнес-логика	В Service-слое	В Handler-классе	EventHandler	
Канонический пример	Spring Guides REST	Spring Framework docs	Spring Data REST Guide	

2.7.1 @RestController (аннотационный подход)

Классический подход, применяемый в Spring Guides [4]. Контроллер аннотируется @RestController, маршруты определяются через @GetMapping, @PostMapping и т. д. Обработка запроса делегируется сервисному слою. Этот подход является наиболее распространённым в экосистеме Spring и обеспечивает декларативное определение API через аннотации.

Архитектура следует паттерну «контроллер–сервис–репозиторий»: контроллер принимает HTTP-запрос, валидирует DTO, передаёт в сервис, который содержит бизнес-логику и взаимодействует с репозиторием для доступа к данным. Идемпотентность POST обеспечивается внешним HTTP-фильтром (IdempotencyFilter), который перехватывает запрос до попадания в контроллер.

2.7.2 Functional Endpoints (WebMvc.fn)

Функциональный подход из Spring Framework [5]. Маршруты определяются программно через `RouterFunction<ServerResponse>`, а обработчики — через функции `ServerRequest → ServerResponse`. Полностью отсутствуют аннотации маршрутизации.

Ключевое отличие от `@RestController` — в способе композиции маршрутов. Вместо распределения маршрутов по аннотациям на методах, все маршруты определяются в одном месте (`PaymentRouter`) через цепочку вызовов `.GET()`, `.POST()` и т. д. Обработчики (`PaymentHandler`) — это обычные компоненты Spring, не наследующие от каких-либо контроллерных классов. Такой подход обеспечивает лучшую тестируемость отдельных обработчиков в изоляции.

Идемпотентность POST обеспечивается тем же `IdempotencyFilter`, что и для `@RestController`, поскольку фильтры работают на уровне Servlet API, до маршрутизации запроса.

2.7.3 Spring Data REST

Автоматическая генерация CRUD-эндпоинтов из JPA-репозитория [6]. Контроллер не пишется вручную — достаточно аннотировать репозиторий `@RepositoryRestResource`. Spring Data REST автоматически создаёт все CRUD-операции, поддерживает HATEOAS (Hypermedia As The Engine Of Application State), генерирует HAL-формат ответов и предоставляет HAL Explorer для интерактивного исследования API.

Идемпотентность обеспечивается встроенными механизмами:

- `@Version` (optimistic locking) — Hibernate проверяет версию при каждом UPDATE, Spring Data REST генерирует ETag из значения версии и поддерживает заголовки `If-Match` и `If-None-Match`;
- UNIQUE constraints на уровне базы данных предотвращают создание дубликатов;
- `@RepositoryEventHandler` позволяет добавить бизнес-логику через обработчики событий (`HandleBeforeCreate`, `HandleBeforeSave`).

2.8 Средства тестирования

- **JUnit 5** — фреймворк модульного тестирования, поддерживающий параметризованные тесты, вложенные классы и расширения [10].
- **Spring Boot Test** — интеграционное тестирование Spring-приложений с автоконфигурацией контекста [11].

- **MockMvc** — тестирование HTTP-запросов без запуска реального HTTP-сервера, с полной поддержкой фильтров и маршрутизации.
- **Testcontainers** — библиотека для запуска Docker-контейнеров в тестах. Используется singleton-паттерн: один контейнер PostgreSQL запускается при старте первого теста и переиспользуется всеми последующими [7].
- **Spring Profiles** — переключение конфигурации для тестирования. Профиль `test` включает идемпотентность, профиль `test-non-idempotent` отключает её, что позволяет запускать один и тот же набор тестов в двух режимах.

3 Результаты

3.1 Описание разработанного программного обеспечения

Разработано Spring Boot приложение, реализующее REST API для трёх доменов:

- **Заказы** (`/api/orders`) — управление заказами через аннотационный `@RestController`;
- **Платежи** (`/api/payments`) — обработка платежей через функциональные эндпоинты (`RouterFunction`);
- **Товары** (`/api/products`) — каталог товаров через автогенерируемые эндпоинты Spring Data REST.

Стек технологий: Spring Boot 3.3, Java 17, PostgreSQL 16, Flyway, Testcontainers, Docker Compose.

3.1.1 Структура проекта

```
src/main/java/com/example/idempotency/
|-- config/           # Конфигурация (свойства, фильтры)
|-- idempotency/      # Механизм идемпотентности
|-- order/           # @RestController
|-- payment/         # Functional Endpoints
|-- product/         # Spring Data REST
+-- exception/       # Обработка ошибок
```

3.1.2 Схема базы данных

База данных состоит из четырёх таблиц, управляемых через миграции Flyway:

Таблица 4: Схема базы данных

Таблица	Ключевые поля	Constraints	Назначение
orders	id (UUID PK), description, amount, status	UNIQUE(desc, amount)	Заказы
payments	id (UUID PK), order_id (FK), amount, status	FK → orders(id)	Платежи
products	id (BIGSERIAL PK), sku, name, price, version	UNIQUE(sku)	Товары
idempotency_keys	id, key, method, path, response_status, response_body	UNIQUE(key)	Ключи

Таблица `idempotency_keys` содержит поле `expires_at` для TTL ключей (по умолчанию 24 часа). Поле `response_body` хранит сериализованный JSON-ответ для повторного использования.

3.1.3 Компоненты механизма идемпотентности

Механизм идемпотентности реализован через три компонента:

1. **IdempotencyFilter** — HTTP-фильтр (наследник `OncePerRequestFilter`), перехватывающий POST-запросы к энд-поинтам заказов и платежей. Реализует алгоритм 1. Может быть отключён через параметр `app.idempotency.enabled=false`.
2. **IdempotencyService** — сервис для работы с хранилищем ключей. Предоставляет методы `findByKey()`, `registerKey()` и `saveResponse()`. Операции выполняются в транзакциях для обеспечения атомарности.
3. **IdempotencyKeyEntity** — JPA-сущность, представляющая запись в таблице `idempotency_keys`. Содержит ключ, HTTP-метод, путь запроса, статус и тело кэшированного ответа, а также временные метки создания и истечения.

3.1.4 Профили конфигурации

Приложение поддерживает два режима работы через Spring Profiles:

- **Профиль по умолчанию:** идемпотентность включена (`enabled=true`), Flyway применяет миграции V1 и V2, все constraints активны.

- **Профиль non-idempotent:** идемпотентность отключена (`enabled=false`), Flyway дополнительно применяет миграцию V3, которая удаляет UNIQUE constraints на таблицах `orders` и `products`.

Переключение профилей позволяет запускать одни и те же тесты в двух конфигурациях и наглядно демонстрировать последствия отсутствия механизмов идемпотентности.

3.2 Результаты тестирования (профиль idempotent)

Основной набор из 22 интеграционных тестов проходит успешно:

Таблица 5: Результаты тестирования (профиль idempotent)

Тестовый класс	Тестов	Результат	Тип контроллера
OrderIdempotencyTest	6	PASS	@RestController
PaymentIdempotencyTest	5	PASS	Functional Endpoints
ProductIdempotencyTest	5	PASS	Spring Data REST
DataImportTest	2	PASS	Импорт JSON/CSV
NonIdempotentProfileTest	4	PASS	Демонстрация багов
Итого	22	PASS	

3.3 Тест-кейсы по контроллерам

3.3.1 @RestController (OrderIdempotencyTest)

1. **POST с одинаковым Idempotency-Key дважды** — создаётся только один заказ, второй запрос возвращает кэшированный ответ. Проверяется через утверждение `assertEquals(1, count)`.
2. **POST с разными ключами** — создаются два разных заказа, что подтверждает: идемпотентность привязана к конкретному ключу, а не к содержимому запроса.
3. **POST без Idempotency-Key** — возвращается 400 Bad Request, что предотвращает случайное создание неидемпотентных запросов.
4. **PUT идемпотентен** — повторное обновление с теми же данными не изменяет состояние. Проверяется сравнением полей `description`, `amount`, `status` после двух последовательных PUT.
5. **DELETE идемпотентен** — первый вызов возвращает 204 No Content (ресурс удалён), второй — 404 Not Found (ресурс уже отсутствует). Оба варианта соответствуют идемпотентному поведению.

6. **GET идиempотентен** — три последовательных GET-запроса возвращают одинаковый JSON-ответ.

3.3.2 Functional Endpoints (PaymentIdempotencyTest)

1. **POST платежа с одним ключом дважды** — создаётся один платёж. Демонстрирует, что IdempotencyFilter работает с RouterFunction так же, как с @RestController.
2. **POST с разными ключами** — создаются два платежа.
3. **POST без ключа** — 400 Bad Request.
4. **GET идиempотентен** — стабильный результат при многократных запросах.
5. **Retry (3 запроса с одним ключом)** — создаётся один платёж, несмотря на 3 попытки. Это ключевой production-сценарий: клиент повторяет запрос из-за таймаута, но система обрабатывает его только один раз.

3.3.3 Spring Data REST (ProductIdempotencyTest)

1. **POST дубликата SKU** — 409 Conflict. UNIQUE constraint на поле sku предотвращает создание товара с тем же артикулом.
2. **PUT с корректной версией** — успешное обновление, @Version автоматически инкрементируется. Проверяется совпадение version + 1 с актуальной версией.
3. **Concurrent PUT (два обновления со старой версией)** — первое обновление проходит успешно (версия 0 → 1), второе получает 412 Precondition Failed, поскольку передан устаревший ETag. Optimistic locking предотвращает lost update.
4. **PATCH идиempотентен** — повторное применение одного и того же PATCH не изменяет данные (имя товара остаётся тем же после второго PATCH).
5. **GET идиempотентен** — стабильный результат.

3.3.4 Импорт данных (DataImportTest)

1. **Импорт заказов из JSON** — загрузка данных из файла orders-input.json, создание через API, сравнение количества, статусов и сумм с эталонным файлом.

2. **Импорт товаров из CSV** — загрузка данных из файла `products-input.csv` через Jackson CSV, сохранение в БД, затем повторная попытка импорта: все 3 попытки отклоняются `UNIQUE constraint`, количество записей не увеличивается.

3.4 Результаты профиля `non-idempotent`

Профиль `non-idempotent` отключает механизмы идемпотентности:

- `IdempotencyFilter` пропускает все `POST`-запросы без проверки заголовка;
- `Unique constraint` на `orders(description, amount)` удалён миграцией V3;
- `Unique constraint` на `products(sku)` удалён миграцией V3.

Таблица 6: Демонстрация проблем без идемпотентности (профиль `non-idempotent`)

Тест	Обнаруженная проблема	Статус
POST без <code>Idempotency-Key</code>	Запрос проходит без проверки — любой <code>POST</code> создаёт ресурс	BUG
Повторный <code>POST</code> с тем же ключом	Создаётся дубликат заказа — ключ игнорируется	BUG
POST с дубликатом <code>SKU</code>	Создаётся второй товар с тем же артикулом	BUG
Retry платежа (3 раза)	3 платежа вместо 1 — тройное списание средств	BUG

Тесты `NonIdempotentProfileTest` проходят зелёными, подтверждая наличие проблем. Каждый тест использует прямые `assertions`: например, проверяется, что `orderRepository.count() >= 2` — без фильтра действительно создаются дубликаты. Это демонстрирует критическое значение механизмов идемпотентности для корректного функционирования системы.

4 Обсуждение

4.1 Сравнение подходов к обеспечению идемпотентности

Результаты исследования показывают, что каждый тип контроллера Spring требует различного подхода к обеспечению идемпотентности (таблица 7).

Таблица 7: Сравнение механизмов идемпотентности по типам контроллеров

Критерий	@RestController	Functional	Data REST
Механизм POST	Idempotency-Key + фильтр	Idempotency-Key + фильтр	Unique constraints
Механизм PUT	Естественная идемпотентность	Естественная идемпотентность	@Version + ETag
Concurrent защита	На уровне фильтра	На уровне фильтра	Optimistic locking
Сложность реализации	Средняя	Средняя	Низкая (автоматически)
Гибкость	Высокая	Высокая	Ограниченная

4.2 Анализ результатов

Аннотационный @RestController предоставляет наибольшую гибкость в реализации идемпотентности, но требует ручной реализации фильтра и хранилища ключей. Этот подход наиболее распространён в production-системах благодаря явности и читаемости кода. Разработчик полностью контролирует обработку запроса, может добавить произвольную логику валидации, трансформации и обработки ошибок. Недостаток — необходимость писать и поддерживать больше кода.

Функциональные эндпоинты (WebMvc.fn) используют тот же механизм Idempotency-Key через общий фильтр, что демонстрирует важное архитектурное преимущество — фильтры работают на уровне Servlet API, независимо от типа контроллера. Это означает, что один и тот же IdempotencyFilter обеспечивает идемпотентность как для @RestController, так и для RouterFunction. Функциональный стиль обеспечивает лучшую композиционность: маршруты можно комбинировать, вкладывать и переиспользовать программно, в отличие от аннотационного подхода, где маршруты привязаны к методам классов.

Spring Data REST автоматически обеспечивает ряд механизмов идемпотентности: @Version генерирует ETag, а unique constraints предотвращают дубликаты. Это минимизирует объём ручного кода — разработчику достаточно определить JPA-сущность и репозиторий. Однако этот подход ограничивает возможности кастомизации: Idempotency-Key не поддерживается автоматически, а бизнес-логика добавляется через EventHandler, что менее гибко, чем прямое управление в Service-слое.

4.3 Идемпотентность как конвенция: почему нельзя «всё делать через GET»

Изложенные выше свойства HTTP-методов (таблица 1) могут привести на естественный вопрос: если GET и HEAD одновременно безопасны (safe) и идемпотентны, почему бы не использовать их для всех операций? Ответ

принципиально важен для понимания природы идемпотентности: **свойства HTTP-методов — это семантическое соглашение, а не техническое ограничение, накладываемое протоколом.** RFC 9110 [3] определяет ожидаемое поведение методов, но сам по себе протокол не запрещает серверу изменить состояние в ответ на **GET** или проигнорировать идемпотентность **PUT**. Иными словами, *фактическая* идемпотентность и безопасность целиком определяются реализацией на стороне сервера. Тем не менее, отступление от конвенции влечёт за собой ряд серьёзных последствий.

- **Кэширование на промежуточных узлах.** Ответы на **GET** по умолчанию кэшируются браузерами, прямыми и обратными прокси, CDN, service worker'ами. Если **GET** меняет состояние, второй запрос клиента может быть обслужен из кэша и вообще не дойти до сервера — операция «пропадёт». Для **POST** и **PUT** такого кэширования по умолчанию не происходит.
- **Предварительная выборка и префетчинг.** Браузеры, поисковые роботы, антивирусные сканеры ссылок и системы link preview (например, в мессенджерах) свободно вызывают **GET** на любых обнаруженных URL. Если такой запрос порождает побочный эффект, возможны спонтанные изменения данных без ведома пользователя. Исторический пример — эпизод с Google Web Accelerator, который проходил по ссылкам «Удалить», реализованным через **GET**, и удалял записи пользователей.
- **Утечка данных в URL.** Параметры **GET** передаются в строке запроса и попадают в журналы веб-серверов, историю браузера, заголовки **Referer**, аналитические системы. Это делает **GET** непригодным для передачи чувствительных данных (паролей, токенов, платёжных реквизитов).
- **Тело запроса.** Согласно RFC 9110, **GET** формально может содержать тело, но его семантика не определена, и многие прокси, CDN и HTTP-клиенты тело у **GET** удаляют. **HEAD** тела ответа вообще не возвращает и пригоден только для проверки метаданных ресурса.
- **Повторные попытки и отказоустойчивость.** HTTP-клиенты, балансировщики и сервис-мешы автоматически повторяют только те методы, которые спецификация объявляет идемпотентными. Если **GET** имеет побочный эффект, такой автоматический **retry** приведёт к множественному выполнению операции.
- **Наблюдаемость и проверяемость контракта.** API-шлюзы, системы мониторинга, тесты на соответствие REST-конвенциям (напри-

мер, линтеры OpenAPI) исходят из стандартной семантики методов. Нарушение этой семантики делает систему непредсказуемой для сторонних интеграторов.

Таким образом, общепринятое соответствие между методом и типом операции — GET для чтения, POST для создания, PUT для полной замены, PATCH для частичного обновления, DELETE для удаления — является не техническим, а **инженерным требованием**, вытекающим из принципа наименьшего удивления [12]. Разработчик обязан реализовать эти свойства самостоятельно: GET-эндпоинт должен быть read-only, а не «кажущийся» read-only. В этом смысле тестирование идемпотентности — это проверка не протокола, а соответствия реализации декларируемому контракту. Настоящая работа построена именно на этом принципе: для каждого метода проверяется не формальная корректность HTTP, а фактическая инвариантность состояния системы при повторных вызовах.

4.4 Оптимистичная блокировка и границы её применимости для идемпотентности

Механизм @Version, продемонстрированный на примере Spring Data REST для сущности Product, часто преподносится как «решение проблемы идемпотентности». Это упрощение требует уточнения: **версионирование в общем случае не обеспечивает идемпотентность; оно решает связанную, но иную задачу — предотвращение lost update при конкурентных обновлениях** [9; 13].

Различие становится очевидным при рассмотрении повторных запросов. Пусть клиент отправляет PUT /products/42 с заголовком If-Match: "3". Первый запрос успешно меняет версию $v = 3 \rightarrow v = 4$ и возвращает 200 OK. Сетевой таймаут приводит к повторной отправке того же запроса с тем же If-Match: "3". Второй запрос получает 412 Precondition Failed, поскольку текущая версия уже равна 4. С точки зрения строгого определения идемпотентности это *нарушение*: два одинаковых запроса дали разные результаты (200 и 412), несмотря на то что итоговое состояние ресурса идентично. Клиент, получивший 412, вынужден либо интерпретировать этот код как «успех, просто повторно», либо выполнить дополнительный GET для перечитывания версии — что усложняет протокол взаимодействия.

Идемпотентность PUT в нашей системе обеспечивается не аннотацией @Version, а самой семантикой метода PUT «полная замена состояния»: повторная отправка одинакового тела без If-Match приводит к одному и тому же финальному состоянию ресурса. @Version же решает *ортогональную* задачу — разрывание конфликта между двумя разными клиентами, одновременно редактирующими один ресурс на основе устаревшей версии.

Частный случай, в котором версионирование действительно совпадает с идемпотентностью, — это PUT-операции, где запрос содержит ожидаемую версию не в заголовке If-Match, а в теле запроса как часть бизнес-смысла (например, «применить эту дельту к версии 3»). В такой модели повторный запрос с `expectedVersion = 3` после успеха первого вернёт тот же самый результат «ожидаемая версия устарела» для обоих повторов, что формально идемпотентно. Именно так работают многие системы event sourcing и CQRS. В нашей работе подобная схема *не* используется: `@Version` на `Product` приведён исключительно как демонстрация того, что **Spring Data REST предоставляет optimistic locking «из коробки»** через заголовки ETag/If-Match, чтобы показать разнообразие механизмов, которые Spring-экосистема относит к тематике «идемпотентности и повторяемости запросов». В production-системе, где действительно требуется идемпотентность PUT при retry, следует применять не `@Version`, а паттерн Idempotency-Key [2; 8] — именно он сохраняет *ответ* первого выполнения и возвращает его без повторного исполнения операции.

Таким образом, `@Version` и Idempotency-Key — дополняющие, а не взаимозаменяемые механизмы: первый защищает от *двух разных* запросов от разных клиентов, второй — от *одного и того же* запроса, повторённого из-за сбоя сети.

4.5 Значимость Testcontainers в тестировании идемпотентности

Использование Testcontainers вместо in-memory баз данных (H2) имеет критическое значение для тестирования идемпотентности. Ряд механизмов зависит от особенностей конкретной СУБД:

- **UNIQUE constraints** — поведение при нарушении уникальности различается между H2 и PostgreSQL (разные коды ошибок, разная обработка транзакций).
- **Flyway-миграции** — синтаксис SQL может отличаться (например, `gen_random_uuid()` специфичен для PostgreSQL).
- **Конкурентный доступ** — PostgreSQL поддерживает уровни изоляции и блокировки, отличные от H2.

Singleton-паттерн Testcontainers (один контейнер на все тесты) обеспечивает баланс между точностью и скоростью: тесты работают с реальной СУБД, но контейнер запускается только один раз за сессию.

4.6 Рекомендации для production

На основе проведённого исследования сформулированы следующие рекомендации:

1. **Всегда требовать Idempotency-Key для POST-запросов**, создающих ресурсы или выполняющих побочные эффекты. Это предотвращает дублирование при retry и является отраслевым стандартом [2].
2. **Использовать optimistic locking (@Version)** для защиты от конкурентных обновлений. Это особенно важно для систем с высокой нагрузкой.
3. **Устанавливать TTL** для ключей идемпотентности. Рекомендуемое значение — 24 часа [8]. Слишком короткий TTL не защитит от поздних retry, слишком длинный — приведёт к росту таблицы ключей.
4. **Хранить ключи** в той же СУБД, что и бизнес-данные, для обеспечения транзакционной консистентности. Альтернатива (Redis) обеспечивает лучшую производительность, но вводит риск рассогласования.
5. **Тестировать с реальной СУБД** через Testcontainers [7], а не с in-memory заменителями, для достоверности результатов.
6. **Комбинировать механизмы**: Idempotency-Key для POST, @Version для PUT, UNIQUE constraints как последний рубеж защиты от дубликатов.

5 Заключение

В данной работе проведено исследование и демонстрация тестирования идемпотентности REST API на базе Spring Boot. Разработано приложение с тремя кардинально различными типами контроллеров, для каждого из которых реализованы и протестированы механизмы обеспечения идемпотентности.

Основные результаты:

- Формализована математическая модель идемпотентности HTTP-запросов с доказательством корректности паттерна Idempotency-Key.
- Реализован и протестирован паттерн Idempotency-Key с хранением ключей в PostgreSQL, применимый к любому типу контроллера через Servlet-фильтр.

- Продемонстрированы три кардинально различных подхода к организации REST API в Spring: аннотационный (@RestController), функциональный (RouterFunction) и автогенерируемый (Spring Data REST), каждый с собственным механизмом обеспечения идемпотентности.
- Разработано 22 интеграционных теста с использованием Testcontainers и PostgreSQL, покрывающих все аспекты идемпотентности для трёх типов контроллеров.
- Продемонстрировано критическое значение механизмов идемпотентности: при их отключении (профиль non-idempotent) система допускает дублирование данных и тройное списание при retry.

6 Инструкция по установке и запуску

6.1 Требования

- Java 17+ (JDK)
- Maven 3.8+
- Docker (для PostgreSQL и Testcontainers)

6.2 Запуск приложения

1. Запустить PostgreSQL через Docker Compose:

```
docker-compose up -d
```

2. Собрать и запустить приложение:

```
mvn spring-boot:run
```

3. API доступно по адресу `http://localhost:8080`:

- `/api/orders` — заказы (@RestController)
- `/api/payments` — платежи (Functional Endpoints)
- `/api/products` — товары (Spring Data REST)
- `/api/explorer` — HAL Explorer (Spring Data REST)

6.3 Запуск тестов

1. Запуск всех тестов (профиль с идемпотентностью):

```
mvn test
```

2. Запуск тестов без идемпотентности (демонстрация багов):

```
mvn test -Dtest="NonIdempotentProfileTest"
```

Примечание: для запуска тестов требуется Docker, так как Testcontainers автоматически запускает PostgreSQL в контейнере.

6.4 Пример использования API

Создание заказа с Idempotency-Key:

```
curl -X POST http://localhost:8080/api/orders \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: unique-key-123" \
  -d '{"description": "Ноутбук", "amount": 89999.99}'
```

Повторный запрос с тем же ключом вернёт кэшированный ответ (заказ не дублируется).

Список литературы

1. *Helland P.* Idempotence Is Not a Medical Condition // Communications of the ACM. Т. 55. — ACM, 2012. — С. 56—65.
2. *Stripe.* Idempotent Requests — Stripe API. — 2024. — URL: https://stripe.com/docs/api/idempotent_requests ; Stripe API Documentation.
3. *Fielding R., Nottingham M., Reschke J.* HTTP Semantics. — 2022. — URL: <https://www.rfc-editor.org/rfc/rfc9110> ; Internet Engineering Task Force. RFC 9110.
4. *Spring Team.* Building REST services with Spring. — 2024. — URL: <https://spring.io/guides/tutorials/rest> ; Spring Guides.
5. *Spring Framework.* Functional Endpoints — Spring Web MVC. — 2024. — URL: <https://docs.spring.io/spring-framework/reference/web/webmvc-functional.html> ; Spring Framework Reference Documentation.
6. *Spring Team.* Accessing JPA Data with REST. — 2024. — URL: <https://spring.io/guides/gs/accessing-data-rest> ; Spring Guides.
7. *AtomicJar.* Testcontainers — Integration testing with real dependencies. — 2024. — URL: <https://testcontainers.com/> ; Официальная документация.
8. *Dalal S., Jena J.* The Idempotency-Key HTTP Header Field. — 2024. — URL: <https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/> ; draft-ietf-httpapi-idempotency-key-header-06. IETF Internet-Draft.
9. *Fowler M.* Patterns of Enterprise Application Architecture. — Addison-Wesley, 2002. — ISBN 978-0321127426.
10. *Beck K.* Test Driven Development: By Example. — Addison-Wesley, 2002. — ISBN 978-0321146533.
11. *Spring Team.* Testing in Spring Boot. — 2024. — URL: <https://docs.spring.io/spring-boot/reference/testing/index.html> ; Spring Boot Reference Documentation.
12. *Richardson L., Amundsen M., Ruby S.* RESTful Web APIs. — O'Reilly Media, 2013. — ISBN 978-1449358068.
13. *Kleppmann M.* Designing Data-Intensive Applications. — O'Reilly Media, 2017. — ISBN 978-1449373320.

Приложение А. Листинги исходного кода

А.1. docker-compose.yml

Листинг 1: docker-compose.yml — PostgreSQL для разработки

```
1 services:
2   postgres:
3     image: postgres:16-alpine
4     container_name: idempotency-postgres
5     environment:
6       POSTGRES_DB: idempotency_demo
7       POSTGRES_USER: demo
8       POSTGRES_PASSWORD: demo
9     ports:
10      - "5432:5432"
11     volumes:
12      - pgdata:/var/lib/postgresql/data
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U demo -d
15         idempotency_demo"]
16       interval: 5s
17       timeout: 3s
18       retries: 5
19 volumes:
20   pgdata:
```

А.2. application.yml

Листинг 2: application.yml — основная конфигурация

```
1 spring:
2   application:
3     name: idempotency-demo
4   datasource:
5     url: jdbc:postgresql://localhost:5432/idempotency_demo
6     username: demo
7     password: demo
8     driver-class-name: org.postgresql.Driver
9   jpa:
10     hibernate:
11       ddl-auto: validate
12     open-in-view: false
13     properties:
14       hibernate:
15         format_sql: true
16   flyway:
17     enabled: true
18     locations: classpath:db/migration
```

```

19 data:
20   rest:
21     base-path: /api
22
23 app:
24   idempotency:
25     enabled: true
26     key-ttl-hours: 24
27
28 server:
29   port: 8080

```

A.3. application-non-idempotent.yml

Листинг 3: application-non-idempotent.yml — профиль без идемпотентности

```

1 app:
2   idempotency:
3     enabled: false
4
5 spring:
6   flyway:
7     locations: classpath:db/migration,classpath:db/migration-
8               non-idempotent
9   jpa:
10    properties:
11      hibernate:
12        '[jakarta.persistence.lock.timeout]': 0

```

A.4. V1__init__schema.sql

Листинг 4: V1__init__schema.sql — основная схема БД

```

1 -- Таблица заказов (домен OrderController)
2 CREATE TABLE orders (
3   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4   description VARCHAR(255) NOT NULL,
5   amount DECIMAL(19, 2) NOT NULL,
6   status VARCHAR(50) NOT NULL DEFAULT 'CREATED',
7   created_at TIMESTAMP NOT NULL DEFAULT NOW(),
8   CONSTRAINT uk_order_description_amount UNIQUE (description,
9   amount)
10 );
11
12 -- Таблица платежей (домен PaymentRouter)
13 CREATE TABLE payments (
14   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
15   order_id UUID NOT NULL REFERENCES orders(id),
16   amount DECIMAL(19, 2) NOT NULL,

```

```

16     status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
17     created_at TIMESTAMP NOT NULL DEFAULT NOW()
18 );
19
20 -- Таблица товаров (домен ProductRepository)
21 CREATE TABLE products (
22     id BIGSERIAL PRIMARY KEY,
23     sku VARCHAR(100) NOT NULL UNIQUE,
24     name VARCHAR(255) NOT NULL,
25     price DECIMAL(19, 2) NOT NULL,
26     quantity INTEGER NOT NULL DEFAULT 0,
27     version INTEGER NOT NULL DEFAULT 0
28 );

```

A.5. V2__idempotency_keys.sql

Листинг 5: V2__idempotency_keys.sql — таблица ключей идиempотентности

```

1 CREATE TABLE idempotency_keys (
2     id BIGSERIAL PRIMARY KEY,
3     idempotency_key VARCHAR(255) NOT NULL UNIQUE,
4     method VARCHAR(10) NOT NULL,
5     path VARCHAR(500) NOT NULL,
6     response_status INTEGER,
7     response_body TEXT,
8     created_at TIMESTAMP NOT NULL DEFAULT NOW(),
9     expires_at TIMESTAMP NOT NULL DEFAULT NOW() + INTERVAL '24
    hours'
10 );
11
12 CREATE INDEX idx_idempotency_key ON idempotency_keys(
    idempotency_key);
13 CREATE INDEX idx_idempotency_expires ON idempotency_keys(
    expires_at);

```

A.6. V3__drop_constraints.sql

Листинг 6: V3__drop_constraints.sql — удаление ограничений (профиль non-idempotent)

```

1 ALTER TABLE orders DROP CONSTRAINT IF EXISTS
    uk_order_description_amount;
2 ALTER TABLE products DROP CONSTRAINT IF EXISTS products_sku_key
    ;

```

A.7. IdempotencyApplication.java

Листинг 7: IdempotencyApplication.java — точка входа

```
1 package com.example.idempotency;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.
    SpringApplication;
5
6 @SpringBootApplication
7 public class IdempotencyApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(IdempotencyApplication.class,
10                                args);
11     }
12 }
```

A.8. IdempotencyProperties.java

Листинг 8: IdempotencyProperties.java — настройки идемпотентности

```
1 package com.example.idempotency.config;
2
3 import org.springframework.boot.context.properties.
    ConfigurationProperties;
4
5 @ConfigurationProperties(prefix = "app.idempotency")
6 public class IdempotencyProperties {
7     private boolean enabled = true;
8     private int keyTtlHours = 24;
9
10    public boolean isEnabled() { return enabled; }
11    public void setEnabled(boolean enabled) { this.enabled =
        enabled; }
12    public int getKeyTtlHours() { return keyTtlHours; }
13    public void setKeyTtlHours(int keyTtlHours) { this.
        keyTtlHours = keyTtlHours; }
14 }
```

A.9. WebMvcConfig.java

Листинг 9: WebMvcConfig.java — регистрация фильтра

```
1 package com.example.idempotency.config;
2
3 import com.example.idempotency.idempotency.IdempotencyFilter;
4 import com.example.idempotency.idempotency.IdempotencyService;
5 import org.springframework.boot.context.properties.
    EnableConfigurationProperties;
6 import org.springframework.boot.web.servlet.
    FilterRegistrationBean;
```

```

7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9
10 @Configuration
11 @EnableConfigurationProperties(IdempotencyProperties.class)
12 public class WebMvcConfig {
13     @Bean
14     public FilterRegistrationBean<IdempotencyFilter>
15         idempotencyFilterRegistration(
16             IdempotencyService idempotencyService,
17             IdempotencyProperties properties) {
18         var reg = new FilterRegistrationBean<IdempotencyFilter>
19             >();
20         reg.setFilter(new IdempotencyFilter(idempotencyService,
21             properties));
22         reg.addUrlPatterns("/api/orders/*", "/api/orders",
23             "/api/payments/*", "/api/payments");
24         reg.setOrder(1);
25         return reg;
26     }
27 }

```

A.10. IdempotencyKeyEntity.java

Листинг 10: IdempotencyKeyEntity.java — сущность ключа идемпотентности

```

1 package com.example.idempotency.idempotency;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "idempotency_keys")
8 public class IdempotencyKeyEntity {
9     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
10     private Long id;
11
12     @Column(name = "idempotency_key", nullable = false, unique
13         = true)
14     private String idempotencyKey;
15
16     @Column(nullable = false, length = 10)
17     private String method;
18
19     @Column(nullable = false, length = 500)
20     private String path;
21
22     @Column(name = "response_status")
23     private Integer responseStatus;
24 }

```

```

23
24 @Column(name = "response_body", columnDefinition = "TEXT")
25 private String responseBody;
26
27 @Column(name = "created_at", nullable = false, updatable =
28         false)
29 private LocalDateTime createdAt = LocalDateTime.now();
30
31 @Column(name = "expires_at", nullable = false)
32 private LocalDateTime expiresAt = LocalDateTime.now().
33         plusHours(24);
34
35 public IdempotencyKeyEntity() {}
36 public IdempotencyKeyEntity(String key, String method,
37         String path) {
38         this.idempotencyKey = key; this.method = method; this.
39         path = path;
40 }
41
42 // Getters u Setters
43 public Long getId() { return id; }
44 public String getIdempotencyKey() { return idempotencyKey;
45         }
46 public Integer getResponseStatus() { return responseStatus;
47         }
48 public void setResponseStatus(Integer s) { this.
49         responseStatus = s; }
50 public String getResponseBody() { return responseBody; }
51 public void setResponseBody(String b) { this.responseBody =
52         b; }
53 }

```

A.11. IdempotencyKeyRepository.java

Листинг 11: IdempotencyKeyRepository.java

```

1 package com.example.idempotency.idempotency;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import java.util.Optional;
5
6 public interface IdempotencyKeyRepository
7         extends JpaRepository<IdempotencyKeyEntity, Long> {
8         Optional<IdempotencyKeyEntity> findByIdempotencyKey(String
9                 key);
10 }

```

A.12. IdempotencyService.java

Листинг 12: IdempotencyService.java — сервис управления ключами

```
1 package com.example.idempotency.idempotency;
2
3 import org.springframework.stereotype.Service;
4 import org.springframework.transaction.annotation.Transactional;
5
6 import java.util.Optional;
7
8 @Service
9 public class IdempotencyService {
10     private final IdempotencyKeyRepository repository;
11
12     public IdempotencyService(IdempotencyKeyRepository
13         repository) {
14         this.repository = repository;
15     }
16
17     public Optional<IdempotencyKeyEntity> findByKey(String key)
18     {
19         return repository.findByIdempotencyKey(key);
20     }
21
22     @Transactional
23     public IdempotencyKeyEntity registerKey(String key, String
24         method, String path) {
25         return repository.save(new IdempotencyKeyEntity(key,
26             method, path));
27     }
28
29     @Transactional
30     public void saveResponse(String key, int status, String
31         responseBody) {
32         repository.findByIdempotencyKey(key).ifPresent(entity
33             -> {
34                 entity.setResponseStatus(status);
35                 entity.setResponseBody(responseBody);
36                 repository.save(entity);
37             });
38     }
39 }
```

A.13. IdempotencyFilter.java

Листинг 13: IdempotencyFilter.java — HTTP-фильтр идиempотентности

```
1 package com.example.idempotency.idempotency;
2
3 import com.example.idempotency.config.IdempotencyProperties;
4 import jakarta.servlet.*;
5 import jakarta.servlet.http.*;
```

```

6 import org.springframework.dao.DataIntegrityViolationException;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.web.filter.OncePerRequestFilter;
9 import org.springframework.web.util.
    ContentCachingResponseWrapper;
10 import java.io.IOException;
11 import java.nio.charset.StandardCharsets;
12
13 public class IdempotencyFilter extends OncePerRequestFilter {
14     public static final String IDEMPOTENCY_KEY_HEADER = "
        Idempotency-Key";
15     private final IdempotencyService idempotencyService;
16     private final IdempotencyProperties properties;
17
18     public IdempotencyFilter(IdempotencyService svc,
        IdempotencyProperties props) {
19         this.idempotencyService = svc; this.properties = props;
20     }
21
22     @Override
23     protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException {
24         if (!"POST".equalsIgnoreCase(request.getMethod())) {
25             chain.doFilter(request, response); return;
26         }
27         if (!properties.isEnabled()) {
28             chain.doFilter(request, response); return;
29         }
30         String key = request.getHeader(IDEMPOTENCY_KEY_HEADER);
31         if (key == null || key.isBlank()) {
32             response.setStatus(HttpStatus.BAD_REQUEST.value());
33             response.setContentType("application/json;charset=
                UTF-8");
34             response.getWriter().write(
35                 "{\"error\":\"Idempotency-Key required\"}");
36             return;
37         }
38         var existing = idempotencyService.findByKey(key);
39         if (existing.isPresent()) {
40             var cached = existing.get();
41             if (cached.getResponseStatus() != null) {
42                 response.setStatus(cached.getResponseStatus());
43                 response.setContentType("application/json;
                    charset=UTF-8");
44                 if (cached.getResponseBody() != null)
45                     response.getWriter().write(cached.
                        getResponseBody());
46                 return;
47             }
48             response.setStatus(HttpStatus.CONFLICT.value());
49             return;
50         }
51

```



```

52     }
53     try {
54         idempotencyService.registerKey(key,
55             request.getMethod(), request.getRequestURI());
56     } catch (DataIntegrityViolationException e) {
57         response.setStatus(HttpStatus.CONFLICT.value());
58         return;
59     }
60     var wrapper = new ContentCachingResponseWrapper(
61         response);
62     chain.doFilter(request, wrapper);
63     String body = new String(
64         wrapper.getContentAsByteArray(), StandardCharsets.
65             UTF_8);
66     idempotencyService.saveResponse(key, wrapper.getStatus
67         (), body);
68     wrapper.copyBodyToResponse();
69 }

```

A.14. Order.java

Листинг 14: Order.java — сущность заказа

```

1 package com.example.idempotency.order;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5 import java.time.LocalDateTime;
6 import java.util.UUID;
7
8 @Entity @Table(name = "orders")
9 public class Order {
10     @Id @GeneratedValue(strategy = GenerationType.UUID)
11     private UUID id;
12     @Column(nullable = false)
13     private String description;
14     @Column(nullable = false, precision = 19, scale = 2)
15     private BigDecimal amount;
16     @Column(nullable = false) @Enumerated(EnumType.STRING)
17     private OrderStatus status = OrderStatus.CREATED;
18     @Column(name = "created_at", nullable = false, updatable =
19         false)
20     private LocalDateTime createdAt = LocalDateTime.now();
21
22     public Order() {}
23     public Order(String description, BigDecimal amount) {
24         this.description = description; this.amount = amount;
25     }
26     // Getters u Setters
27     public UUID getId() { return id; }

```

```

27     public String getDescription() { return description; }
28     public void setDescription(String d) { this.description = d
        ; }
29     public BigDecimal getAmount() { return amount; }
30     public void setAmount(BigDecimal a) { this.amount = a; }
31     public OrderStatus getStatus() { return status; }
32     public void setStatus(OrderStatus s) { this.status = s; }
33 }

```

A.15. OrderStatus.java

Листинг 15: OrderStatus.java

```

1 package com.example.idempotency.order;
2
3 public enum OrderStatus { CREATED, CONFIRMED, CANCELLED }

```

A.16. OrderDto.java

Листинг 16: OrderDto.java — DTO заказа с валидацией

```

1 package com.example.idempotency.order;
2
3 import jakarta.validation.constraints.*;
4 import java.math.BigDecimal;
5
6 public record OrderDto(
7     @NotBlank(message = "Описание обязательно") String
        description,
8     @NotNull @DecimalMin("0.01") BigDecimal amount,
9     OrderStatus status
10 ) {}

```

A.17. OrderRepository.java

Листинг 17: OrderRepository.java

```

1 package com.example.idempotency.order;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import java.util.UUID;
5
6 public interface OrderRepository extends JpaRepository<Order,
    UUID> {}

```

A.18. OrderService.java

ЛИСТИНГ 18: OrderService.java — бизнес-логика заказов

```
1 package com.example.idempotency.order;
2
3 import org.springframework.stereotype.Service;
4 import org.springframework.transaction.annotation.Transactional
5 ;
6 import java.util.*;
7
8 @Service
9 public class OrderService {
10     private final OrderRepository orderRepository;
11
12     public OrderService(OrderRepository repo) { this.
13         orderRepository = repo; }
14
15     public List<Order> findAll() { return orderRepository.
16         findAll(); }
17
18     public Order findById(UUID id) {
19         return orderRepository.findById(id)
20             .orElseThrow(() -> new OrderNotFoundException(id));
21     }
22
23     @Transactional
24     public Order create(OrderDto dto) {
25         return orderRepository.save(new Order(dto.description()
26             , dto.amount()));
27     }
28
29     @Transactional
30     public Order update(UUID id, OrderDto dto) {
31         var order = findById(id);
32         order.setDescription(dto.description());
33         order.setAmount(dto.amount());
34         if (dto.status() != null) order.setStatus(dto.status())
35         ;
36         return orderRepository.save(order);
37     }
38
39     @Transactional
40     public void delete(UUID id) { orderRepository.delete(
41         findById(id)); }
```

```
37 public static class OrderNotFoundException extends
38     RuntimeException {
39     public OrderNotFoundException(UUID id) {
40         super("Order not found: " + id);
41     }
42 }
```

A.19. OrderController.java

Листинг 19: OrderController.java — тип контроллера №1: @RestController

```

1 package com.example.idempotency.order;
2
3 import jakarta.validation.Valid;
4 import org.springframework.http.*;
5 import org.springframework.web.bind.annotation.*;
6 import java.util.*;
7
8 @RestController
9 @RequestMapping("/api/orders")
10 public class OrderController {
11     private final OrderService orderService;
12
13     public OrderController(OrderService svc) { this.
        orderService = svc; }
14
15     @GetMapping
16     public List<Order> list() { return orderService.findAll();
        }
17
18     @GetMapping("/{id}")
19     public Order get(@PathVariable UUID id) { return
        orderService.findById(id); }
20
21     @PostMapping
22     public ResponseEntity<Order> create(@Valid @RequestBody
        OrderDto dto) {
23         return ResponseEntity.status(HttpStatus.CREATED)
24             .body(orderService.create(dto));
25     }
26
27     @PutMapping("/{id}")
28     public Order update(@PathVariable UUID id, @Valid
        @RequestBody OrderDto dto) {
29         return orderService.update(id, dto);
30     }
31
32     @DeleteMapping("/{id}") @ResponseStatus(HttpStatus.
        NO_CONTENT)
33     public void delete(@PathVariable UUID id) { orderService.
        delete(id); }
34 }

```

A.20. Payment.java

Листинг 20: Payment.java — сущность платежа

```
1 package com.example.idempotency.payment;
2
3 import com.example.idempotency.order.Order;
4 import jakarta.persistence.*;
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7 import java.util.UUID;
8
9 @Entity @Table(name = "payments")
10 public class Payment {
11     @Id @GeneratedValue(strategy = GenerationType.UUID)
12     private UUID id;
13     @ManyToOne(fetch = FetchType.EAGER)
14     @JoinColumn(name = "order_id", nullable = false)
15     private Order order;
16     @Column(nullable = false, precision = 19, scale = 2)
17     private BigDecimal amount;
18     @Column(nullable = false) @Enumerated(EnumType.STRING)
19     private PaymentStatus status = PaymentStatus.PENDING;
20     @Column(name = "created_at", nullable = false, updatable =
21         false)
22     private LocalDateTime createdAt = LocalDateTime.now();
23
24     public Payment() {}
25     public Payment(Order order, BigDecimal amount) {
26         this.order = order; this.amount = amount;
27     }
28     // Getters u Setters
29     public UUID getId() { return id; }
30     public BigDecimal getAmount() { return amount; }
31     public PaymentStatus getStatus() { return status; }
```

A.21. PaymentStatus.java

Листинг 21: PaymentStatus.java

```
1 package com.example.idempotency.payment;
2
3 public enum PaymentStatus { PENDING, COMPLETED, FAILED }
```

A.22. PaymentDto.java

Листинг 22: PaymentDto.java

```

1 package com.example.idempotency.payment;
2
3 import java.math.BigDecimal;
4 import java.util.UUID;
5
6 public record PaymentDto(UUID orderId, BigDecimal amount) {}

```

A.23. PaymentRepository.java

Листинг 23: PaymentRepository.java

```

1 package com.example.idempotency.payment;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import java.util.UUID;
5
6 public interface PaymentRepository extends JpaRepository<
    Payment, UUID> {}

```

A.24. PaymentService.java

Листинг 24: PaymentService.java — бизнес-логика платежей

```

1 package com.example.idempotency.payment;
2
3 import com.example.idempotency.order.*;
4 import org.springframework.stereotype.Service;
5 import org.springframework.transaction.annotation.Transactional;
6 import java.util.*;
7
8 @Service
9 public class PaymentService {
10     private final PaymentRepository paymentRepository;
11     private final OrderRepository orderRepository;
12
13     public PaymentService(PaymentRepository pr, OrderRepository
        or) {
14         this.paymentRepository = pr; this.orderRepository = or;
15     }
16
17     public List<Payment> findAll() { return paymentRepository.
        findAll(); }
18
19     public Payment findById(UUID id) {
20         return paymentRepository.findById(id)
21             .orElseThrow(() -> new PaymentNotFoundException(id)
22             );
23     }
24 }

```

```

23
24     @Transactional
25     public Payment create(PaymentDto dto) {
26         var order = orderRepository.findById(dto.orderId())
27             .orElseThrow(() -> new OrderService.
28                 OrderNotFoundException(dto.orderId()));
29         return paymentRepository.save(new Payment(order, dto.
30             amount()));
31     }
32
33     public static class PaymentNotFoundException extends
34         RuntimeException {
35         public PaymentNotFoundException(UUID id) {
36             super("Payment not found: " + id);
37         }
38     }
39 }

```

A.25. PaymentHandler.java

Листинг 25: PaymentHandler.java — тип контроллера №2: HandlerFunction

```

1 package com.example.idempotency.payment;
2
3 import com.fasterxml.jackson.databind.ObjectMapper;
4 import org.springframework.stereotype.Component;
5 import org.springframework.web.servlet.function.*;
6 import java.net.URI;
7 import java.util.UUID;
8
9 @Component
10 public class PaymentHandler {
11     private final PaymentService paymentService;
12     private final ObjectMapper objectMapper;
13
14     public PaymentHandler(PaymentService svc, ObjectMapper om)
15     {
16         this.paymentService = svc; this.objectMapper = om;
17     }
18
19     public ServerResponse list(ServerRequest request) {
20         return ServerResponse.ok().body(paymentService.findAll
21             ());
22     }
23
24     public ServerResponse getById(ServerRequest request) {
25         var id = UUID.fromString(request.pathVariable("id"));
26         try {
27             return ServerResponse.ok().body(paymentService.
28                 findById(id));
29         } catch (PaymentService.PaymentNotFoundException e) {

```

```

27         return ServerResponse.notFound().build();
28     }
29 }
30
31 public ServerResponse create(ServerRequest request) throws
Exception {
32     var dto = objectMapper.readValue(
33         request.servletRequest().getInputStream(),
34         PaymentDto.class);
35     var payment = paymentService.create(dto);
36     return ServerResponse
37         .created(URI.create("/api/payments/" + payment.
38             getId()))
39         .body(payment);

```

A.26. PaymentRouter.java

Листинг 26: PaymentRouter.java — тип контроллера №2: RouterFunction

```

1 package com.example.idempotency.payment;
2
3 import org.springframework.context.annotation.*;
4 import org.springframework.web.servlet.function.*;
5 import static org.springframework.http.MediaType.
6     APPLICATION_JSON;
7 import static org.springframework.web.servlet.function.
8     RequestPredicates.*;
9
10 @Configuration
11 public class PaymentRouter {
12     @Bean
13     public RouterFunction<ServerResponse> paymentRoutes(
14         PaymentHandler h) {
15         return RouterFunctions.route()
16             .path("/api/payments", b -> b
17                 .GET("", accept(APPLICATION_JSON), h::list)
18                 .GET("/{id}", accept(APPLICATION_JSON), h::
19                     getId)
20                 .POST("", contentType(APPLICATION_JSON), h::
21                     create)
22             ).build();
23     }
24 }

```

A.27. Product.java

Листинг 27: Product.java — сущность с @Version для optimistic locking

```
1 package com.example.idempotency.product;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5
6 @Entity @Table(name = "products")
7 public class Product {
8     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long id;
10    @Column(nullable = false, unique = true, length = 100)
11    private String sku;
12    @Column(nullable = false)
13    private String name;
14    @Column(nullable = false, precision = 19, scale = 2)
15    private BigDecimal price;
16    @Column(nullable = false)
17    private Integer quantity = 0;
18    @Version
19    private Integer version;
20
21    public Product() {}
22    public Product(String sku, String name, BigDecimal price,
23        Integer qty) {
24        this.sku = sku; this.name = name; this.price = price;
25        this.quantity = qty;
26    }
27    // Getters u Setters
28    public Long getId() { return id; }
29    public String getSku() { return sku; }
30    public String getName() { return name; }
31    public void setName(String n) { this.name = n; }
32    public Integer getQuantity() { return quantity; }
33    public void setQuantity(Integer q) { this.quantity = q; }
34    public Integer getVersion() { return version; }
35 }
```

A.28. ProductRepository.java

Листинг 28: ProductRepository.java — тип контроллера №3: Spring Data REST

```
1 package com.example.idempotency.product;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.data.rest.core.annotation.
5     RepositoryRestResource;
6 import java.util.Optional;
```

```

7 @RepositoryRestResource(collectionResourceRel = "products",
   path = "products")
8 public interface ProductRepository extends JpaRepository<
   Product, Long> {
9     Optional<Product> findBySku(String sku);
10 }

```

A.29. ProductEventHandler.java

Листинг 29: ProductEventHandler.java — обработчик событий Spring Data REST

```

1 package com.example.idempotency.product;
2
3 import org.slf4j.*;
4 import org.springframework.data.rest.core.annotation.*;
5 import org.springframework.stereotype.Component;
6
7 @Component @RepositoryEventHandler
8 public class ProductEventHandler {
9     private static final Logger log =
10         LoggerFactory.getLogger(ProductEventHandler.class);
11
12     @HandleBeforeCreate
13     public void handleBeforeCreate(Product product) {
14         log.info("Creating product: SKU={}", product.getSku());
15         if (product.getQuantity() == null) product.setQuantity
16             (0);
17     }
18
19     @HandleBeforeSave
20     public void handleBeforeSave(Product product) {
21         log.info("Updating product: id={}, version={}",
22             product.getId(), product.getVersion());
23     }
24 }

```

A.30. GlobalExceptionHandler.java

Листинг 30: GlobalExceptionHandler.java — глобальная обработка ошибок

```

1 package com.example.idempotency.exception;
2
3 import com.example.idempotency.order.OrderService;
4 import com.example.idempotency.payment.PaymentService;
5 import org.springframework.dao.DataIntegrityViolationException;
6 import org.springframework.http.*;
7 import org.springframework.orm.
   ObjectOptimisticLockingFailureException;

```

```

8 import org.springframework.web.bind.
   MethodArgumentNotValidException;
9 import org.springframework.web.bind.annotation.*;
10 import java.time.LocalDateTime;
11 import java.util.Map;
12
13 @RestControllerAdvice
14 public class GlobalExceptionHandler {
15
16     @ExceptionHandler(OrderService.OrderNotFoundException.class
17     )
18     public ResponseEntity<Map<String, Object>>
19         handleOrderNotFound(
20             OrderService.OrderNotFoundException e) {
21         return buildError(HttpStatus.NOT_FOUND, e.getMessage())
22             ;
23     }
24
25     @ExceptionHandler(PaymentService.PaymentNotFoundException.
26         class)
27     public ResponseEntity<Map<String, Object>>
28         handlePaymentNotFound(
29             PaymentService.PaymentNotFoundException e) {
30         return buildError(HttpStatus.NOT_FOUND, e.getMessage())
31             ;
32     }
33
34     @ExceptionHandler(DataIntegrityViolationException.class)
35     public ResponseEntity<Map<String, Object>>
36         handleDataIntegrity(
37             DataIntegrityViolationException e) {
38         return buildError(HttpStatus.CONFLICT, "Duplicate
39             resource");
40     }
41
42     @ExceptionHandler(ObjectOptimisticLockingFailureException.
43         class)
44     public ResponseEntity<Map<String, Object>>
45         handleOptimisticLock(
46             ObjectOptimisticLockingFailureException e) {
47         return buildError(HttpStatus.CONFLICT, "Version
48             conflict");
49     }
50
51     @ExceptionHandler(MethodArgumentNotValidException.class)
52     public ResponseEntity<Map<String, Object>> handleValidation
53         (
54             MethodArgumentNotValidException e) {
55         var msg = e.getBindingResult().getFieldErrors().stream
56             ()
57             .findFirst().map(err -> err.getField() + ": "

```

```

45         + err.getDefaultMessage()).orElse("Validation
           error");
46     return buildError(HttpStatus.BAD_REQUEST, msg);
47 }
48
49 private ResponseEntity<Map<String, Object>> buildError(
50     HttpStatus status, String message) {
51     return ResponseEntity.status(status).body(Map.of(
52         "timestamp", LocalDateTime.now().toString(),
53         "status", status.value(),
54         "error", status.getReasonPhrase(),
55         "message", message));
56 }
57 }

```

A.31. BaseIntegrationTest.java

Листинг 31: BaseIntegrationTest.java — базовый класс с Testcontainers

```

1 package com.example.idempotency;
2
3 import org.springframework.boot.test.autoconfigure.web.servlet.
   AutoConfigureMockMvc;
4 import org.springframework.boot.test.context.SpringBootTest;
5 import org.springframework.test.context.*;
6 import org.testcontainers.containers.PostgreSQLContainer;
7
8 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.
   RANDOM_PORT)
9 @AutoConfigureMockMvc
10 @ActiveProfiles("test")
11 public abstract class BaseIntegrationTest {
12
13     static final PostgreSQLContainer<?> POSTGRES =
14         new PostgreSQLContainer<>("postgres:16-alpine")
15             .withDatabaseName("idempotency_test")
16             .withUsername("test").withPassword("test");
17
18     static { POSTGRES.start(); }
19
20     @DynamicPropertySource
21     static void configureProperties(DynamicPropertyRegistry
   registry) {
22         registry.add("spring.datasource.url", POSTGRES::
   getJdbcUrl);
23         registry.add("spring.datasource.username", POSTGRES::
   getUsername);
24         registry.add("spring.datasource.password", POSTGRES::
   getPassword);
25     }
26 }

```
